

Icebreaker

Natural language control of a Linux operating system

Icebreaker is an Ubuntu-based AI operating system project that translates user requests into structured, auditable system operations. It combines language models with explicit policy checks and a sandboxed execution layer to make system administration more accessible and controllable.

Problem and practical value

Routine Linux work requires users to know commands, parameters and the consequences of changing system state. Icebreaker accepts intent in natural language and routes supported tasks through a controlled workflow. The design focuses on execution transparency, human oversight and traceable outcomes.

Implemented capabilities

Area	Examples of supported tool families
Files	List, read, inspect, write and delete files, subject to tool and path policy.
System operations	Query or manage packages and services; inspect processes; read CPU, memory and disk information.
User interfaces	GTK4 desktop applications and a Textual terminal interface connected to a Python daemon.
Distribution	Ubuntu image construction, systemd services and build tooling for amd64 and arm64.

Architecture at a glance

User request → Intent model → Controller checks → Local execution model
→ Tool validation and verifier → Rust daemon → Result and audit

The Quarantined Brain interprets requests and proposes a structured intent. The Privileged Brain proposes the execution call. The Controller mediates the transition, applying schemas, risk policy and approval before dispatch. Models generate proposals; their outputs remain subject to validation.

Technology foundation

Python Controller • Rust mcpd daemon • JSON Schema and JSON-RPC • GTK4 and Textual • llama-server inference • Linux Landlock and seccomp • Docker-assisted ISO builds • systemd service management.

Deployment model. The default intent backend is Gemini 2.5 Flash and requires cloud connectivity. The Privileged Brain runs locally. Backend configuration supports alternative providers and local inference; the default configuration is therefore a hybrid deployment.

Technical basis: dual-brain/controller/main.py and backends/; cx-distro/distro/controller.toml; dual-brain/gui/ and terminal/; src/mcpd/src/; cx-distro/ build tooling.

Controlled system execution

Policy checks and kernel restrictions surround model output

The central engineering principle is separation of responsibilities: language models interpret or propose; the Controller applies policy; the execution daemon validates and performs the operation. This limits the authority granted to any model-generated response.

Control layers

Layer	Implemented responsibility
Intent validation	JSON Schema checks the structured request. Controller QB backend requests exclude execution tools.
Risk policy	The classifier can escalate risk above the rule-based floor; it cannot lower that floor.
Human oversight	Higher-risk actions can require approval. Covered destructive operations use an impact simulation and an explicit commit step.
Call validation	Constrained output, per-tool schema checks, bounded retries and a verifier evaluate the proposed call before dispatch.
Execution restrictions	Rust mcpd uses stdio JSON-RPC. Linux Landlock constrains filesystem access and seccomp restricts system calls.
Audit integrity	Controller logs use append-only writes, per-record fsync and a hash chain to support reconstruction and tamper detection.

Illustrative destructive operation

For a covered file deletion, the system resolves the requested target, classifies its risk and produces an impact preview. Approval permits a validated operation to reach the execution daemon. A denied request stops the workflow; the outcome is recorded. The preview is simulation-based, not a claim that every action runs in an overlay filesystem.

Current implementation boundaries

The Privileged Brain receives a typed envelope containing a tool schema and, where relevant, target, content and hint fields. The original “opaque UUID only” description is narrower than the current implementation. These fields remain untrusted input to subsequent checks.

The repository demonstrates implementation and integration work. It does not establish enterprise fleet scale, a universal safety guarantee or security certification. Runtime behavior depends on the selected edition, configuration and host environment.

Sources: dual-brain/controller/{main.py,session.py,risk_classifier.py,audit.py}; src/mcpd/src/{server.rs,tools/cow.rs,sandbox/}; docs/2026-08-03_whitepaper_vs_reality_audit.md, including its later status updates.